[9]

Three-Way Merges

In Chapter 8, you learned about cloning and what it's like to collaborate with another person on a Git project. You walked through an example of a friend contributing work to the Rainbow project, pushing the work to the remote repository, and you fetching and merging the work in order to keep your repositories in sync.
All of the merges you have done up until this point in the Rainbow project have been fast-forward merges. In this chapter, you will learn how to carry out a three-way merge. In the process, you will also go over an example of defining upstream branches and you will learn about what happens when you edit files multiple times in your working directory between commits. Finally, you will learn about pulling from a remote repository, and we will discuss how pulling is different from fetching.

State of the Local and Remote Repositories

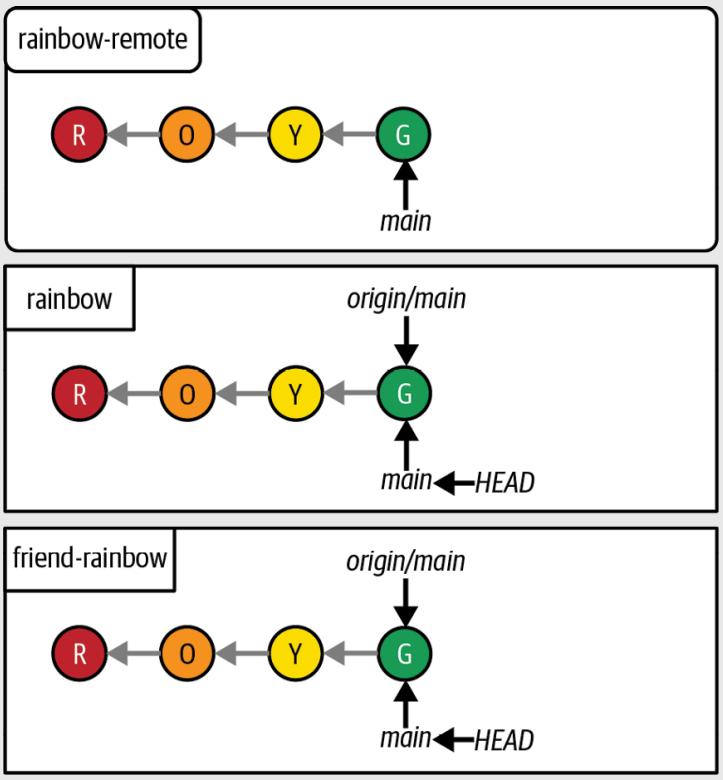
At the start of this chapter, you should have two local repositories called rainbow and friend-rainbow and one remote repository called rainbow-remote. All three of these repositories should be in sync; in other words, they should contain the same commits and branches. I recommend that you continue using two separate text editor windows and command line windows for the rainbow repository and the friend-rainbow repository as you work through the examples in this chapter, as discussed in "The Collaboration Simulation" on page 115.

Visualize it 9-1 shows the state of the local and remote repositories in the Rainbow project with all the commits that were made from Chapter 1 through Chapter 8.

To focus on the commits you are going to make in this chapter, from here on I will simplify the Visualize It diagrams and show only the last two commits that are part of the

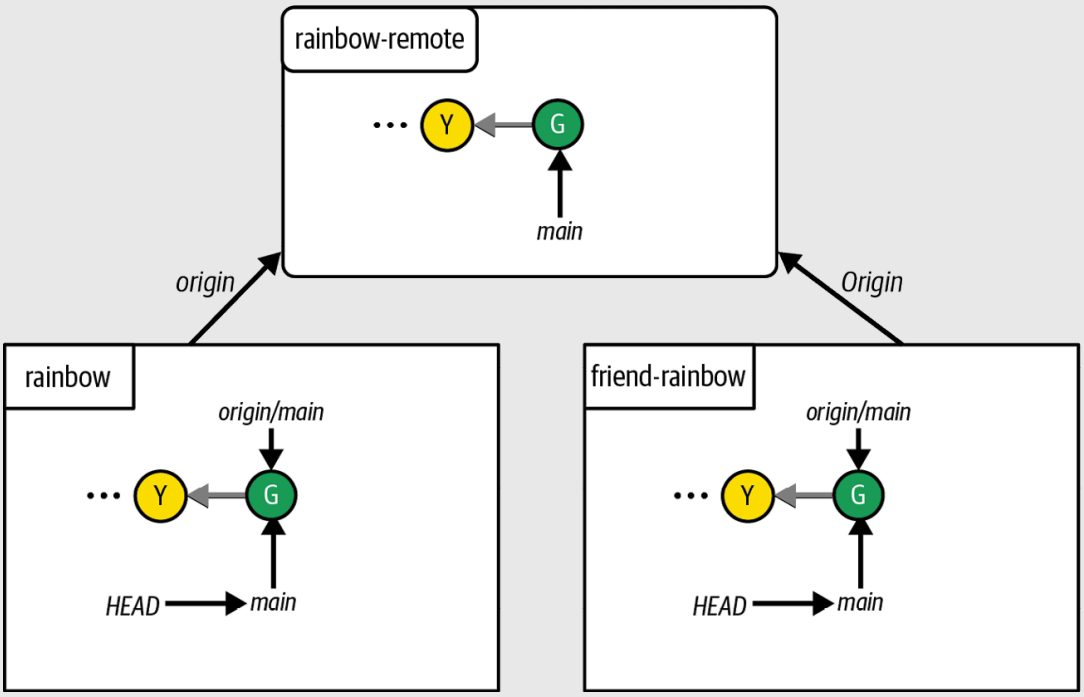
main branch in all the repositories, which are the yellow commit and the green commit. This representation is shown in Visualize it 9-2.

[VISUALIZE IT 9-1]



The Rainbow project at the start of Chapter 9 with all the commits since Chapter 1

[VISUALIZE IT 9-2]



A simplified representation of the Rainbow project at the start of Chapter 9, showing just the last two commits on the main branch in all the repositories

Why Are Three-Way Merges Important?

In Chapter 5, I explained that merging involves integrating the changes made in one branch, called the source branch, into another branch, called the target branch. I also introduced two types of merges: fast-forward merges and three-way merges. Up until now, you've only carried out fast-forward merges, but it is important to learn about three-way merges as well because they are a regular part of a Git user's day-to-day activities.

Three-way merges are a bit more complicated than fast-forward merges because they create merge commits and they may lead to merge conflicts. *Merge conflicts* arise when you merge two branches where different changes have been made to the same parts in the same file(s), or if in one branch a file was deleted that was edited in the other branch.

In Chapter 10, we will discuss merge conflicts in more depth and go over an example of a three-way merge with a merge conflict. In the three-way merge example in this chap-

ter, your friend will edit a different file than the one you will edit, and therefore you won't have any merge conflicts.

As you learned in Chapter 5, three-way merges occur when the development histories of the branches involved in the merge have diverged—in other words, when it is *not* possible to reach the target branch by following the commit history (parents links) of the source branch. Let's explore an example of how this situation might arise in Example Book Project 9-1.

EXAMPLE BOOK PROJECT 9-1

Suppose that while working on the Book project, my coathor and I both decide to make a branch off the main branch at the same time, to work on different chapters. I make the chapter_five branch and my coauthor makes the chapter_six branch, as seen in Figure 9-1.

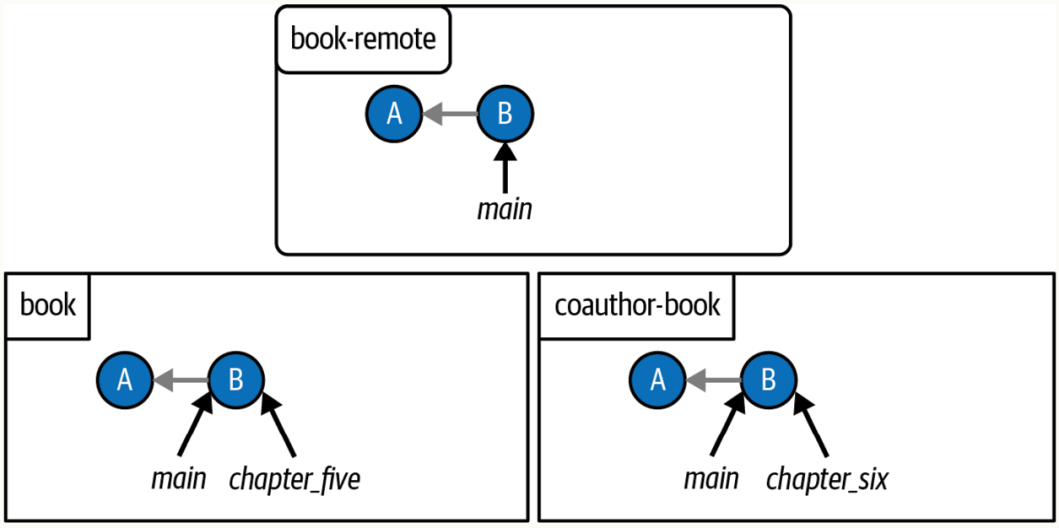


FIGURE 9-1

The Book project after my coauthor and I make branches to work on different chapters

We each work independently on our chapters. My coauthor finishes their work on the chapter_six branch first and proceeds to merge their work into the main branch and push the updated main branch to the remote repository. In Figure 9-2, I represent the work my coathor did as commit C and show the state of all the repositories.

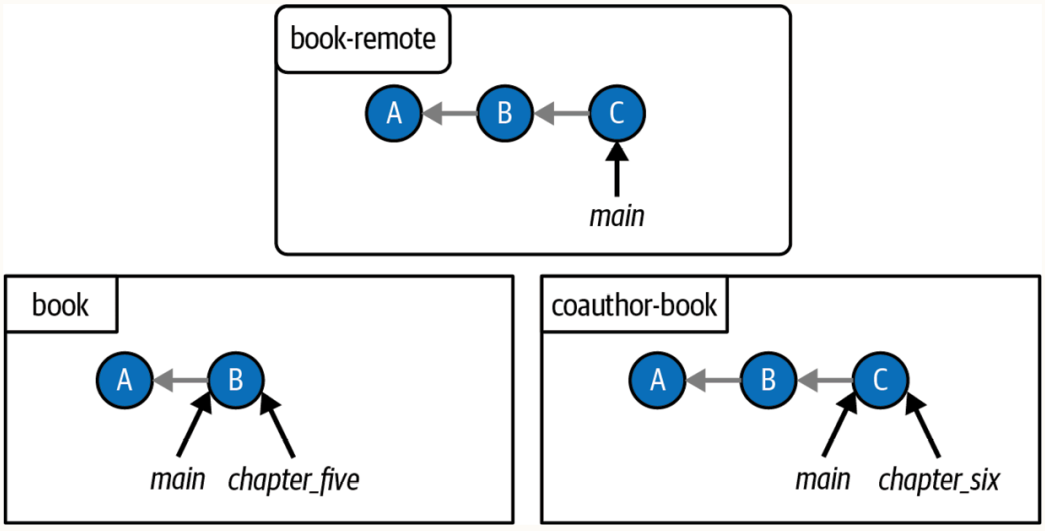


FIGURE 9-2

The Book project after my coauthor pushes their changes to the remote repository

When I finish working on the chapter_five branch, which I will represent as commit D, I also want to merge my work into the main branch. But my coauthor lets me know that they've already added work to the remote main branch, so I first need to update my local main branch with the work that my coauthor added to the remote main branch, as seen in Figure 9-3.

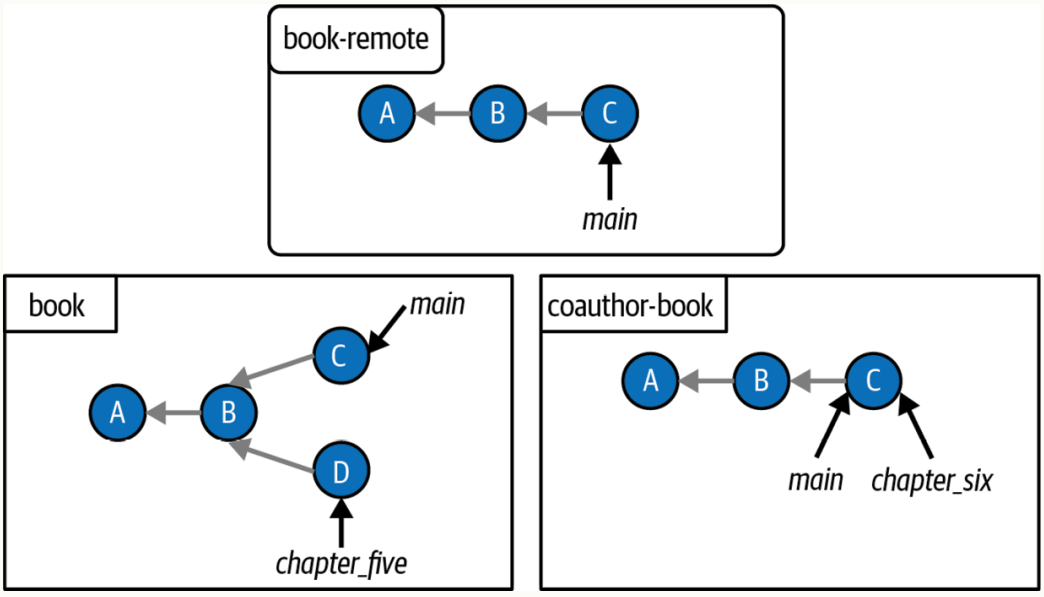


FIGURE 9-3

The Book project after I contribute work to my local chapter_five branch and fetch and integrate the changes from the remote main branch

In Figure 9-3, you can see that the development history of the local main branch in the book repository is made up of commits A, B, and C, while the development history of the chapter_five branch is made up of commits A, B, and D. Since it is not possible to follow the development history of the chapter_five branch to reach the main branch, this means the development histories of these branches have diverged. Next, I have a couple of options.

One option is to merge my local chapter_five branch into the local main branch, which will be a three-way merge, and then push the updated main branch to the remote repository.

The other option is to carry out a merge in the remote repository through a hosting service feature called a *pull request*. You'll learn about pull requests in Chapter 12; for the purposes of this chapter, let's assume I decide to go ahead with the first option and carry out a three-way merge in my local repository.

Figure 9-4 illustrates the state of the repositories when I integrate the chapter_five branch into main through a three-way merge. The merge commit produced by the three-

way merge is represented as commit M.

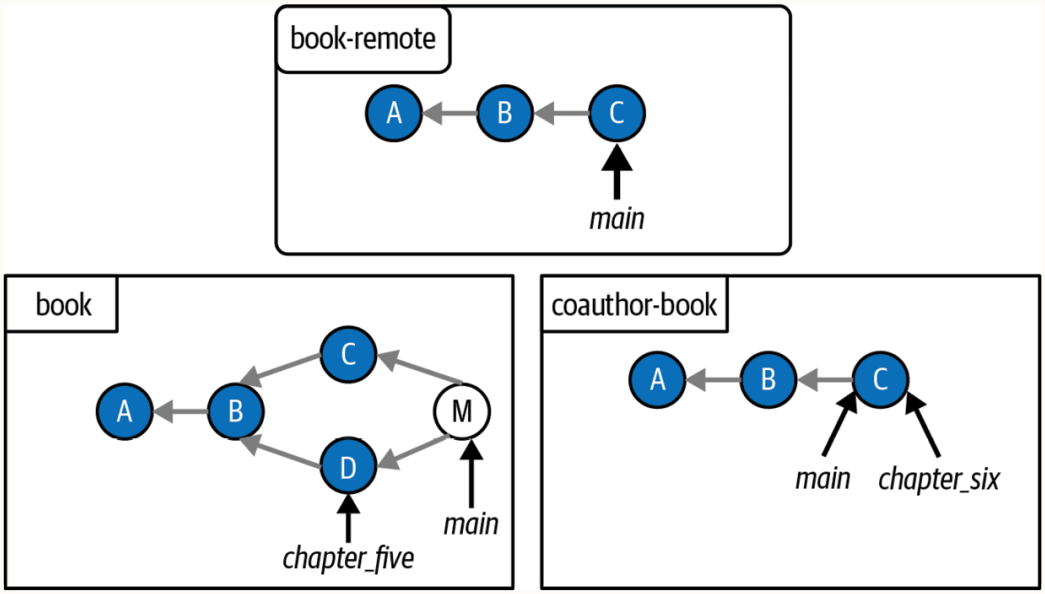


FIGURE 9-4

The Book project after I merge the chapter_five branch into the main branch through a three-way merge

---

As you can see in Example Book Project 9-1, three-way merges produce merge commits, which are commits that can have more than one parent commit.

> [NOTE]
>
> Some people don't like three-way merges because they find that merge commits make the commit history more complicated. To avoid three-way merges you may use the process of rebasing, which you will learn about in Chapter 11.

Next, you'll set up a situation in the Rainbow project where you will have to do a three-way merge. Along the way, you will learn about defining upstream branches and some characteristics of modified files in the working directory.

Setting Up a Three-Way Merge Scenario

First, go to Follow Along 9-1 to start listing colors that are *not* part of the rainbow in a new file called othercolors.txt in the rainbow project directory.

[FOLLOW ALONG 9-1]

1. In the rainbow project directory in your text editor, create a new file called othercolors.txt. Type "Brown is not a color in the rainbow." on line 1 in the file, and save it.

2. rainbow $ **git add othercolors.txt**

3. rainbow $ **git commit -m "brown"**
   [main 7f0a87a] brown
   1 file changed, 1 insertion(+)
   create mode 100644 othercolors.txt

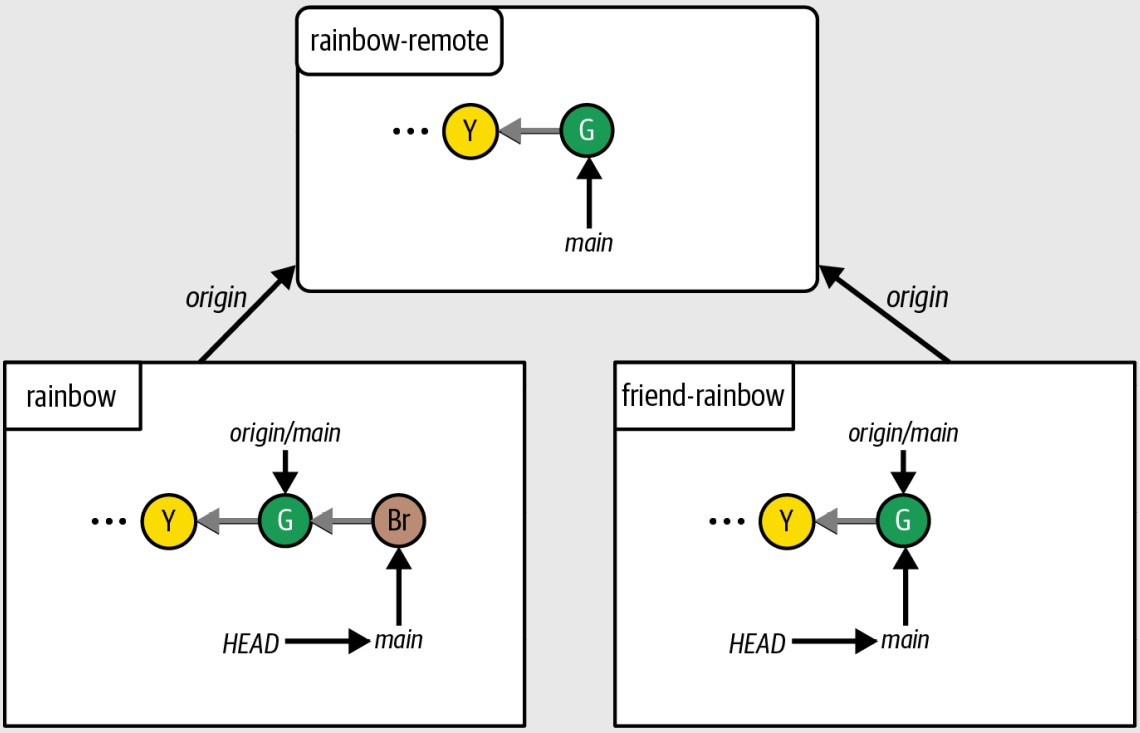
4. rainbow $ **git log**
   commit 7f0a87a318e50638eec50a484bf8dfa76b76d08e (**HEAD -> main**)
   Author: annaskoulikari <gitlearningjourney@gmail.com>
   Date: Sat Feb 19 12:46:29 2022 +0100
   brown
   commit 6987cd2996e245ec24ee9c5ea99874f0a01a31cd (**origin/main**)
   Author: annaskoulikari <gitlearningjourney@gmail.com>
   Date: Sat Feb 19 11:49:03 2022 +0100
   green

What to notice:

- You have made the brown commit in the rainbow repository.

This is illustrated in Visualize it 9-3.

[VISUALIZE IT 9-3]



The Rainbow project after you make the brown commit in the rainbow repository

Next, before you push your commit to the remote repository, let's learn about defining upstream branches.

Defining Upstream Branches

In Chapters 7 and 8, I mentioned that when you push work from a local branch to a remote branch, Git needs a way to know which remote branch you want to push the work to. If no upstream branch is defined for the local branch you're working on, you'll need to specify which remote branch to push to when you enter the git push command. If a local branch has an upstream branch defined for it, you can use git push with no arguments and Git will automatically push the work to that branch.

You also learned that upstream branches are automatically set up when you clone a repository, but *not* when a repository is initialized locally. The rainbow repository was initialized locally, and you have not defined any upstream branches yet.

To avoid specifying the remote repository shortname and the branch every time you use the git push command on the main branch in your rainbow repository, you can define an upstream branch for the main branch and thereafter simply use the git push command with no arguments.

[NOTE]

Once an upstream branch has been defined for a local branch, you can also use other commands, such as git pull, without arguments. You will learn about the git pull command at the end of this chapter.

To set up the upstream branch you will use the git branch command with the -u option, which is short for --set-upstream-to. You'll pass in the name of the remote branch as an argument, specifying the remote repository shortname, a slash, and then the remote branch name (for example, origin/main.) Then, you will witness how you can use the git push command without any additional arguments.

[SAVE THE COMMAND]

git branch -u <shortname>/<branch_name>

Define an upstream branch for the current local branch

To check whether an upstream branch has been defined, you will use the git branch -vv command introduced in Chapter 7.

Go to Follow Along 9-2 to define the upstream branch for the local main branch.

[FOLLOW ALONG 9-2]

1. rainbow $ **git branch -vv**
   * main 7f0a87a brown

2. rainbow $ **git branch -u origin/main**
   branch 'main' set up to track 'origin/main'.

3. rainbow $ **git branch -vv**
   * main 7f0a87a [origin/main: ahead 1] brown

What to notice:

- In step 1, the git branch -vv output shows that there is no upstream branch set for the local main branch.
- In step 2, the command output explicitly states that an upstream branch has been set (branch 'main' set up to track 'origin/main').
- In step 3, the git branch -vv output shows that the main branch from the remote repository with shortname origin has been set as the upstream branch for the local main branch.

Now that you have defined an upstream branch for your local main branch, go to Follow Along 9-3 to use the git push command without any arguments.

[FOLLOW ALONG 9-3]

1. rainbow $ **git push**

```
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 4 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 310 bytes | 310.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0
To github.com:gitlearningjourney/rainbow-remote.git
6987cd2..7f0a87a main -> main
```

2. rainbow $ **git log**

```
commit 7f0a87a318e50638eec50a484bf8dfa76b76d08e (HEAD -> main, origin/main)
Author: annaskoulikari <gitlearningjourney@gmail.com>
Date: Sat Feb 19 12:46:29 2022 +0100
brown
commit 6987cd2996e245ec24ee9c5ea99874f0a01a31cd
Author: annaskoulikari <gitlearningjourney@gmail.com>
Date: Sat Feb 19 11:49:03 2022 +0100
green
```

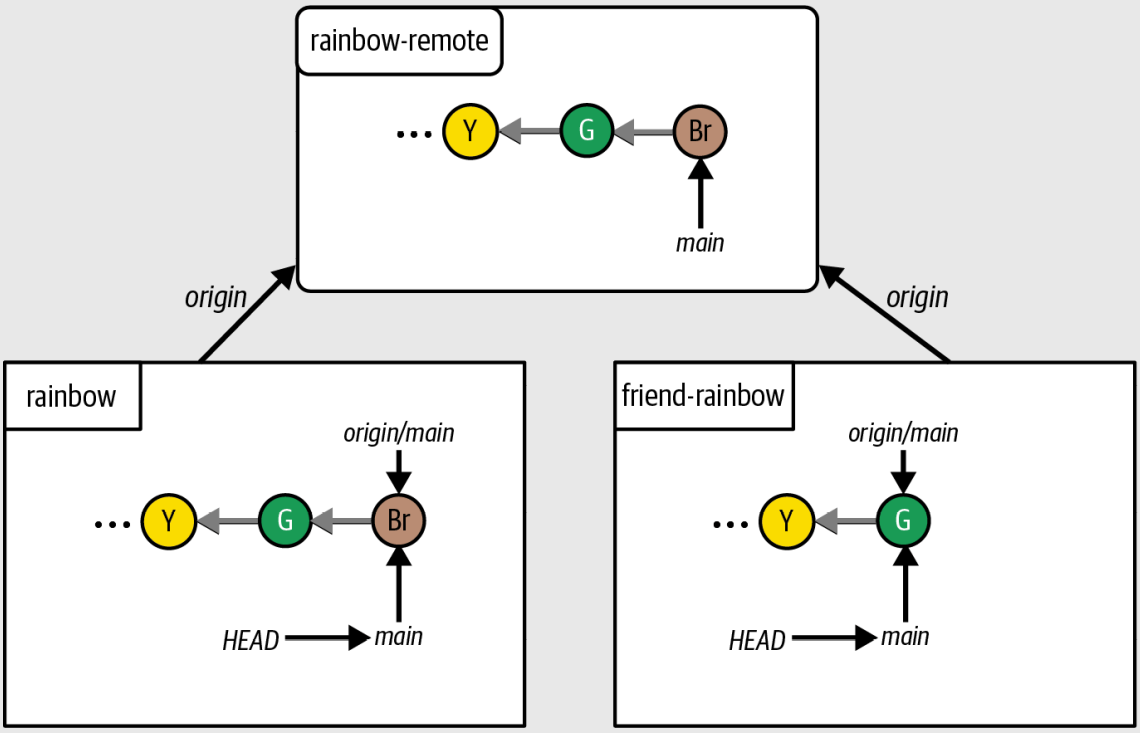
3. Go to the rainbow-remote repository on your hosting service and refresh the page. You should see the brown commit.

What to notice:

- You pushed your local main branch to the remote repository.

This is illustrated in Visualize it 9-4.

[VISUALIZE IT 9-4]



The Rainbow project after you push your local main branch to the rainbow-remote
repository

You have just learned how to define upstream branches in a local repository, and you have made the brown commit in the rainbow repository and pushed it to the remote repository. To end up in a situation where you will have to carry out a three-way merge, there must be divergent development histories between two branches.

Next, your friend will continue working on the local main branch in their local repository *without* fetching the changes you pushed to the remote main branch, which will cause the local main branch in the friend-rainbow repository and the main branch in the rainbow-remote repository to diverge.

While your friend is working on the local main branch, you are also going to learn about some characteristics of modified files in the working directory and what happens when you edit a file multiple times between commits.

Editing the Same File Multiple Times Between Commits

Up until now, you've been editing files just once and adding them to the staging area. However, it is important to understand that if you add a file to the staging area and then make *another change* to the file, Git will interpret this as a new version of the file and it will mark the file as modified. Then, if you want the latest version of the file to be included in your next commit, you will have to add the updated version of the file to the staging area *again*.

To see this in action, in this section your friend is going to add the color blue to the list of colors in rainbowcolors.txt and add the edited file to their staging area—but they will make a typo, writing "Bloo is the fifth color of the rainbow." When they notice this they will edit the file again to fix the typo, and you will see how the file will need to be added to the staging area a second time in order for their latest changes to be included in the next commit.

The upcoming Visualize It diagrams will use the Git Diagram we created in Chapter 2, and we will zoom in on the friend-rainbow repository to illustrate all of this happening. First, go to Follow Along 9-4 to check the state of the friend-rainbow working directory and staging area at the moment.

[FOLLOW ALONG 9-4]

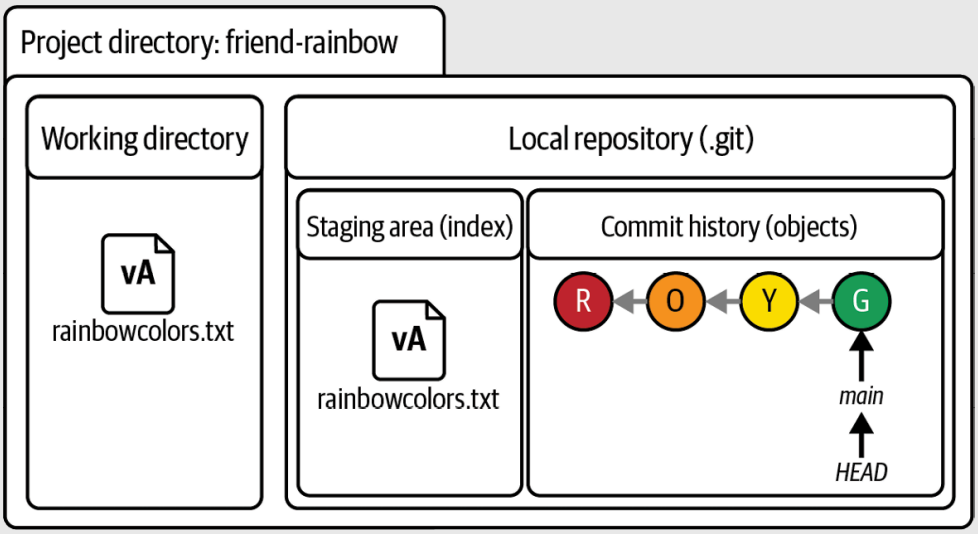
1. friend-rainbow $ **git status**
   On branch main
   Your branch is up to date with 'origin/main'.
   nothing to commit, working tree clean

What to notice:

- The git status output states nothing to commit, working tree clean, which indicates that the state of the working directory and staging area are the same. There are no modified files in the working directory and there are no newly staged files in the staging area.

This observation is illustrated in Visualize it 9-5.

[VISUALIZE IT 9-5]



Git Diagram showing the current state of the friend-rainbow project directory

What to notice:

- The version of the rainbowcolors.txt file in the working directory and staging area mentions the colors red, orange, yellow, and green. We represent it as version A (vA).

Next, in Follow Along 9-5, your friend is going to edit the rainbowcolors.txt file and add the color blue.

[FOLLOW ALONG 9-5]

1. Open the rainbowcolors.txt in the friend-rainbow project directory in a text editor window, add “Bloo is the fifth color of the rainbow.” on line 5, and save the file. Note that you are including the sentence with a typo here on purpose for learning purposes.

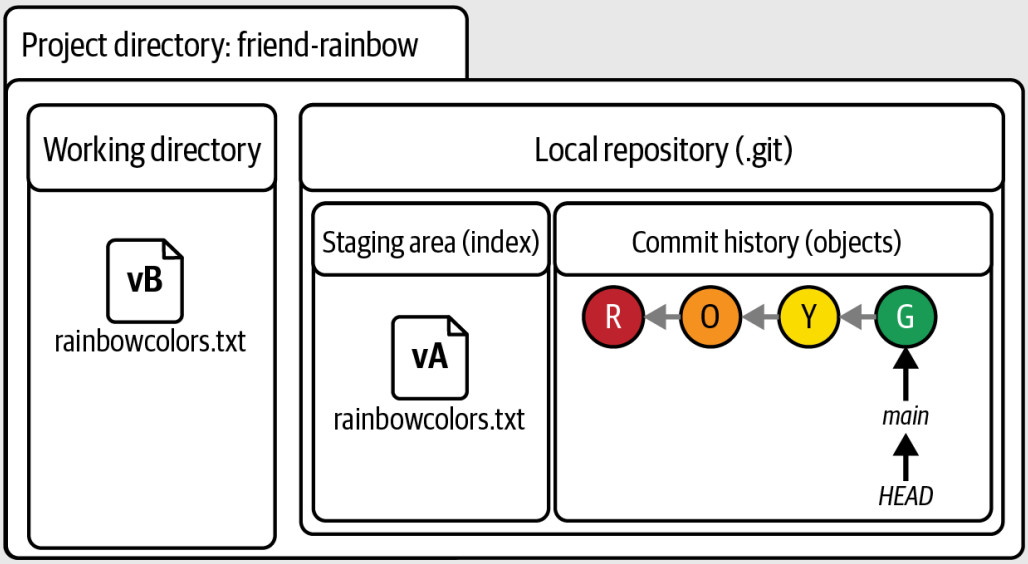
2. friend-rainbow $ **git status**
   On branch main
   Your branch is up to date with 'origin/main'.
   Changes not staged for commit:
   (use "git add <file>..." to update what will be committed)
   (use "git restore <file>..." to discard changes in working directory)
   modified: rainbowcolors.txt
   no changes added to commit (use "git add" and/or "git commit -a")

What to notice:

- In step 2, the git status output indicates that there is now one modified file in the working directory, which is not yet staged for commit.

This observation is illustrated in Visualize it 9-6.

[VISUALIZE IT 9-6]



The friend-rainbow project directory after your friend edits the rainbowcolors.txt file in the working directory

What to notice:

- The version of the rainbowcolors.txt file in the staging area is still vA; it hasn't changed.
- The version of the rainbowcolors.txt file in the working directory has changed: it mentions the colors red, orange, yellow, green, and blue. We represent it as version B (vB).

Next, in Follow Along 9-6, let's see what happens when your friend adds the file to the staging area.

[FOLLOW ALONG 9-6]

1. friend-rainbow $ **git add rainbowcolors.txt**

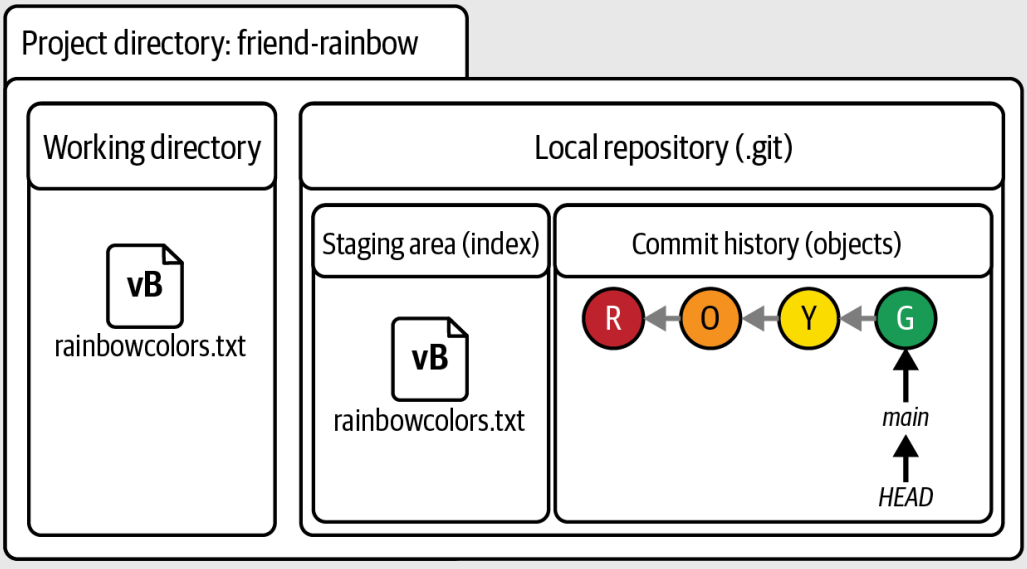
2. friend-rainbow $ **git status**
   On branch main
   Your branch is up to date with 'origin/main'.
   Changes to be committed:
   (use "git restore --staged <file>..." to unstage)
   modified: rainbowcolors.txt

What to notice:

- In step 2, the git status output indicates that there is now one modified file in the working directory that is also staged for commit.

This is illustrated in Visualize it 9-7.

[VISUALIZE IT 9-7]



The friend-rainbow project directory after your friend adds the modified rainbowcolors.txt file to the staging area

What to notice:

- The version of the rainbowcolors.txt file in the staging area has changed from vA to vB.
- The version of the rainbowcolors.txt file in the staging area is the same as the one in the working directory.

Your friend has added the updated version of the rainbowcolors.txt file to the staging area, and it is ready to be committed. Now, go to Follow Along 9-7 to see what happens when your friend goes back to edit the file again to fix the typo, changing it from "Bloo" to "Blue".

[FOLLOW ALONG 9-7]

1. In the friend-rainbow project directory in your text editor, fix the typo on line 5 of the rainbowcolors.txt file so the sentence reads "Blue is the fifth color of the rainbow." Save the file.

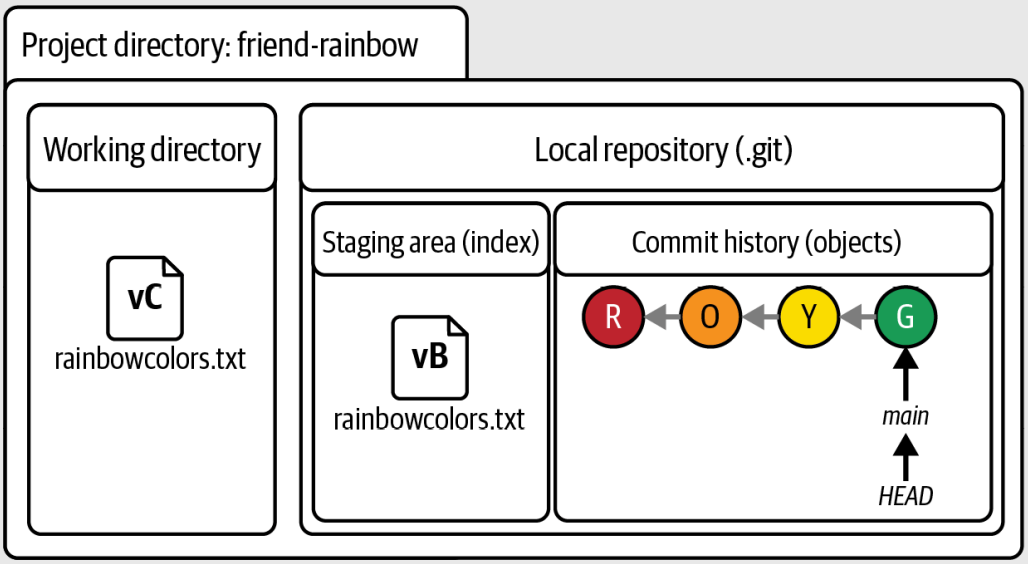
2. friend-rainbow $ **git status**
   On branch main
   Your branch is up to date with 'origin/main'.
   Changes to be committed:
   (use "git restore --staged <file>..." to unstage)
   modified: rainbowcolors.txt
   Changes not staged for commit:
   (use "git add <file>..." to update what will be committed)
   (use "git restore <file>..." to discard changes in working directory)
   modified: rainbowcolors.txt

What to notice:

- In the git status output in step 2, one version of the rainbowcolors.txt file is listed as a modified file that is staged for commit and another version of the rainbowcolors.txt file is listed as a modified file that is *not* staged for commit.

This is illustrated in Visualize it 9-8.

[VISUALIZE IT 9-8]



The friend-rainbow project directory after your friend edits the rainbowcolors.txt file for the second time

What to notice:

- The version of the rainbowcolors.txt file in the staging area is vB. This is the version of the file with the typo.
- The version of the rainbowcolors.txt file in the working directory has changed from vB to version C (vC). The vC file is the version without the typo.

From these observations, you can see that just because a file with a particular name is staged for commit *doesn't mean* it is automatically updated with any other changes you make to it. You need to explicitly add each updated version of a file to the staging area to include the latest changes in your next commit. Go to Follow Along 9-8 to see this in practice.

[FOLLOW ALONG 9-8]

1. friend-rainbow $ **git add rainbowcolors.txt**

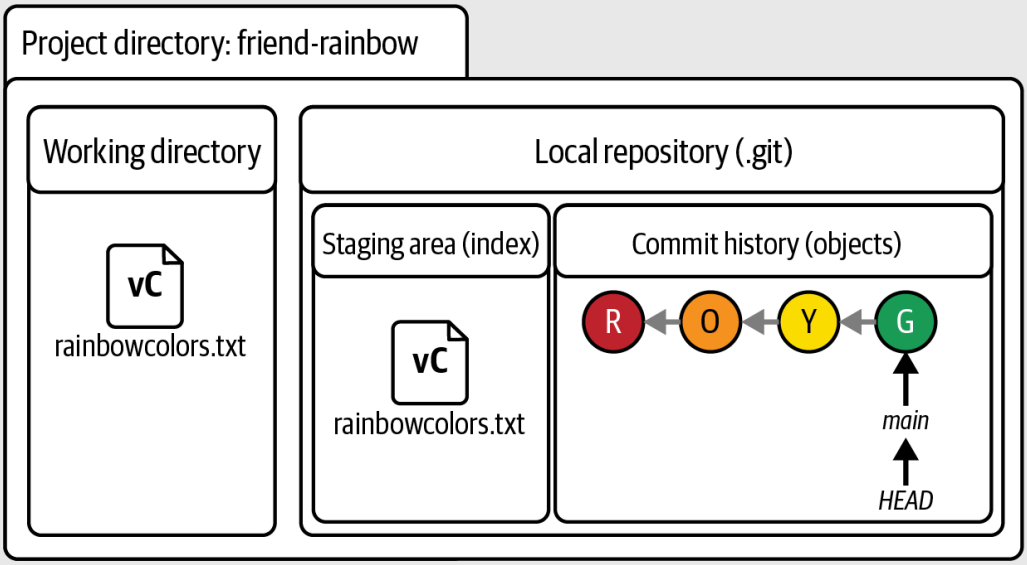
2. friend-rainbow $ **git status**
   On branch main
   Your branch is up to date with 'origin/main'.
   Changes to be committed:
   (use "git restore --staged <file>..." to unstage)
   modified: rainbowcolors.txt

What to notice:

- In the git status output in step 2, the rainbowcolors.txt file is listed as a modified file staged for commit. There are no more modified files that can be added to the staging area.

This is illustrated in Visualize it 9-9.

[VISUALIZE IT 9-9]



The friend-rainbow project directory after your friend adds the rainbowcolors.txt file to the staging area again

What to notice:

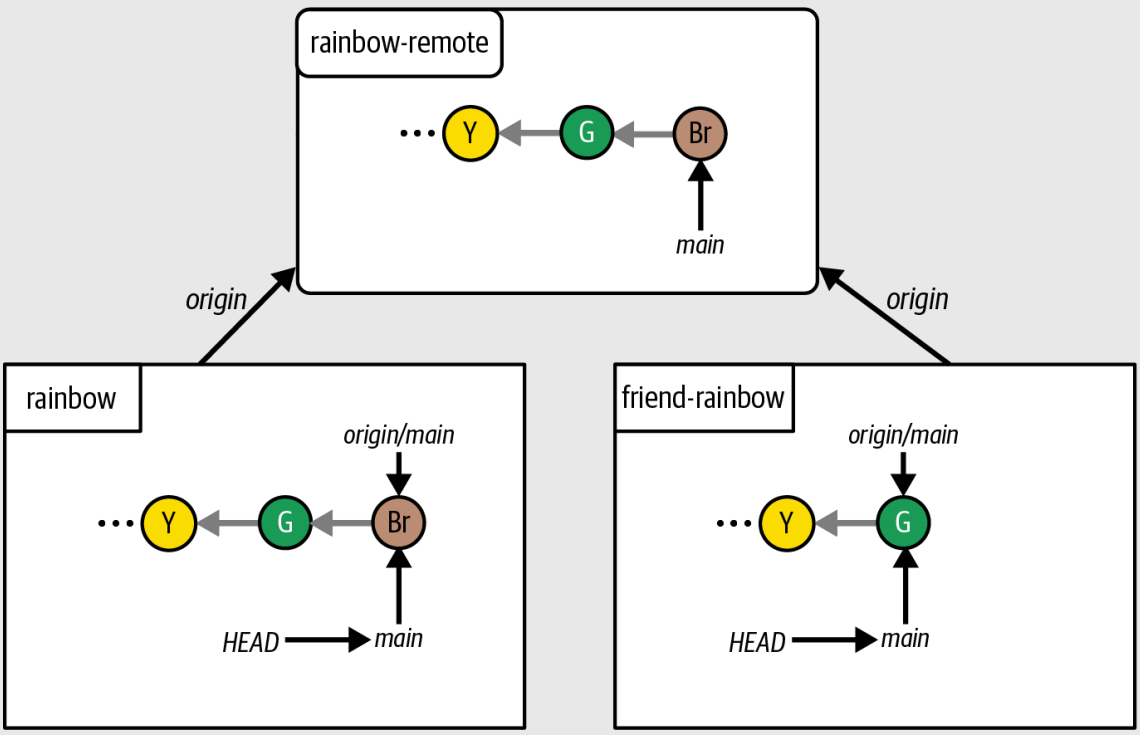
- The version of the rainbowcolors.txt file in the staging area has changed from vB to vC, indicating that the latest version of the rainbowcolors.txt file has been added to the staging area.

Next, your friend is going to finish making the blue commit and attempt to push their work to the remote repository. However, since their local main branch is out of sync with the remote main branch, they will come across an error. You'll see this in the next section.

Working at the Same Time as Others on Different Files

In Visualize it 9-10, you can see that your friend has not fetched the brown commit that you made on the main branch in the rainbow repository and pushed to the remote repository.

[VISUALIZE IT 9-10]



The current state of the local and remote repositories

The local main branch in the friend-rainbow repository is out of sync with the remote main branch. Your friend is ready to make their blue commit and to try to share their work, but when they try to push their work to the remote repository they will get an error. Let's see this in action in Follow Along 9-9.

[FOLLOW ALONG 9-9]

1. friend-rainbow $ **git commit -m "blue"**

```
[main 342bbfc] blue
1 file changed, 2 insertions(+), 1 deletion(-)
```

2. friend-rainbow $ **git push**

```
To github.com:gitlearningjourney/rainbow-remote.git
! [rejected] main -> main (fetch first)
error: failed to push some refs to 'github.com:gitlearningjou
rney/rainbow-remote.git'
hint: Updates were rejected because the remote contains work that you do
hint: not have locally. This is usually caused by hint: another repository
pushing
hint: to the same ref. You may want hint: to first integrate the remote
changes
hint: (e.g., 'git pull hint: ...') before pushing again.
hint: See the 'Note about fast-forwards' in 'git push --help' for details.
```

3. friend-rainbow $ **git log**

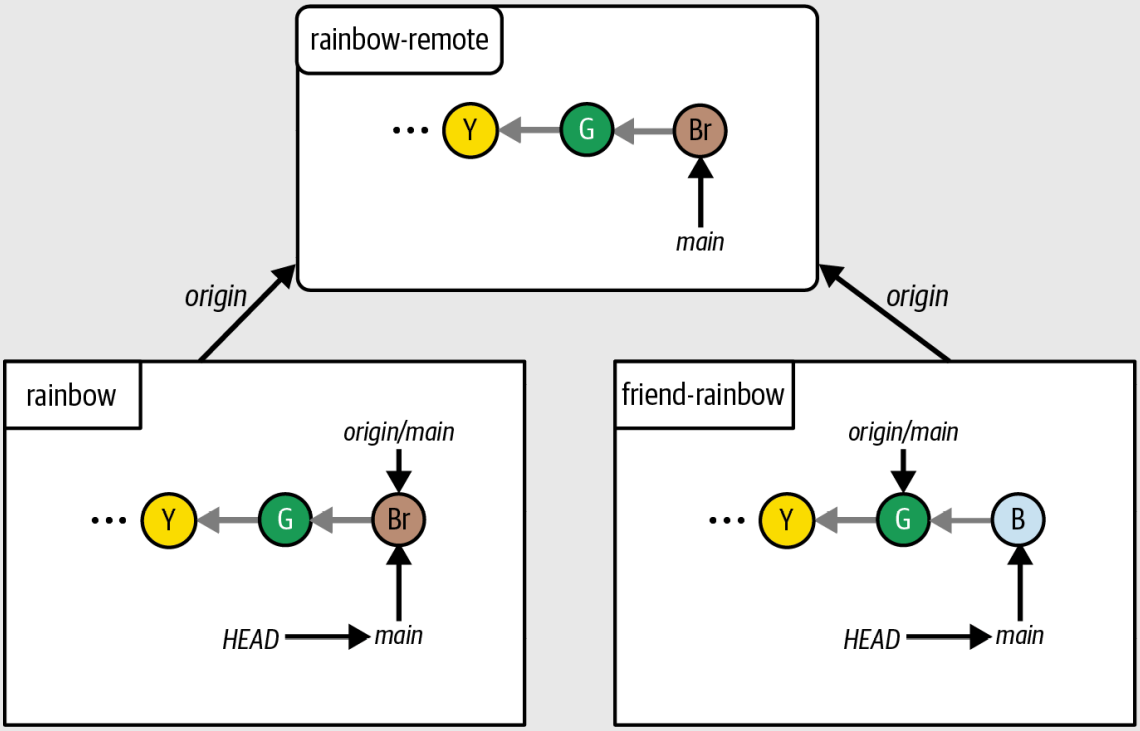
```
commit 342bbfc96bb03053f23ea7f7564ca207c58ceab2 (HEAD -> main)
Author: annaskoulikari <gitlearningjourney@gmail.com>
Date: Sat Feb 19 13:00:56 2022 +0100
blue
commit 6987cd2996e245ec24ee9c5ea99874f0a01a31cd (origin/main,
origin/HEAD)
Author: annaskoulikari <gitlearningjourney@gmail.com>
Date: Sat Feb 19 11:49:03 2022 +0100
green
```

What to notice:

- In step 2, the output of the git push command shows that your friend gets an error. They are not able to push their changes to the remote repository.

The development histories of the local main branch and the remote main branch have diverged, and Git is not able to merge changes from one into the other with a simple fast-forward merge. The addition of the blue commit and the divergent development histories are illustrated in Visualize it 9-11.

[VISUALIZE IT 9-11]



The Rainbow project after your friend makes the blue commit in the friend-rainbow repository

What to notice:

- In the friend-rainbow repository, the local main branch points to the blue commit.
- In the rainbow-remote repository, the main branch points to the brown commit.

Let's take a closer look at the error message your friend received:

error: failed to push some refs to 'https://github.com/gitlearningjourney/rainbow-remote.git' hint: Updates were rejected because the remote contains work that you do not have locally. This is usually caused by another repository pushing to the same ref. You may want to first integrate the remote changes (e.g., 'git pull ...') before pushing again. See the 'Note about fast-forwards' in 'git push --help' for details.

Git is telling your friend that there are commits on the remote main branch that they have not yet fetched (or pulled). It advises your friend that they have to fetch (or pull) the remote work and integrate it into their local main branch before they can push to the remote repository. Note that pulling is similar to fetching, with some differences; we'll cover pulling later in this chapter. For now, to integrate the remote changes, your friend is going to execute a three-way merge.

Three-Way Merge in Practice

In Chapter 8, you learned that incorporating changes from a remote repository is a two-step process:

1. First, you fetch the changes from the remote repository.
2. Second, you integrate the changes into the local branch in the local repository.

Your friend is going to follow the same steps right now, except at step 2, they'll end up carrying out a three-way merge instead of a fast-forward merge because the development histories of the two branches involved in the merge have diverged.

When performing a three-way merge, Git will create a merge commit. To do that, it will enter a text editor in the command line. Let's learn a bit more about how you're going to work with this command line text editor.

INTRODUCING VIM, THE COMMAND LINE TEXT EDITOR

Git, by default, uses a text editor in the command line called *Vim* to write commit messages. If a commit is being made (whether a regular commit or a merge commit) and a commit message is not specified, Git will enter Vim in the command line. You have not yet come across Vim in this book because every time you have made a commit using the git commit command in the Rainbow project you have passed in the -m option and explicitly specified a commit message.

[NOTE]

Git allows you to change the default command line text editor it uses from Vim to another text editor. For example, you can set it to the same text editor you use to edit files in your project.

Vim may appear scary because the command line will look different, and unless you know some Vim basics it is not intuitive to use. Let's cover the necessary basics to navigate Vim.

When Git enters Vim in the command line, you have the choice to enter text, edit text, or approve and save text. In the case of a three-way merge, Git will draft a default commit message for the merge commit and Vim will present it to you, as seen in Figure 9-5.

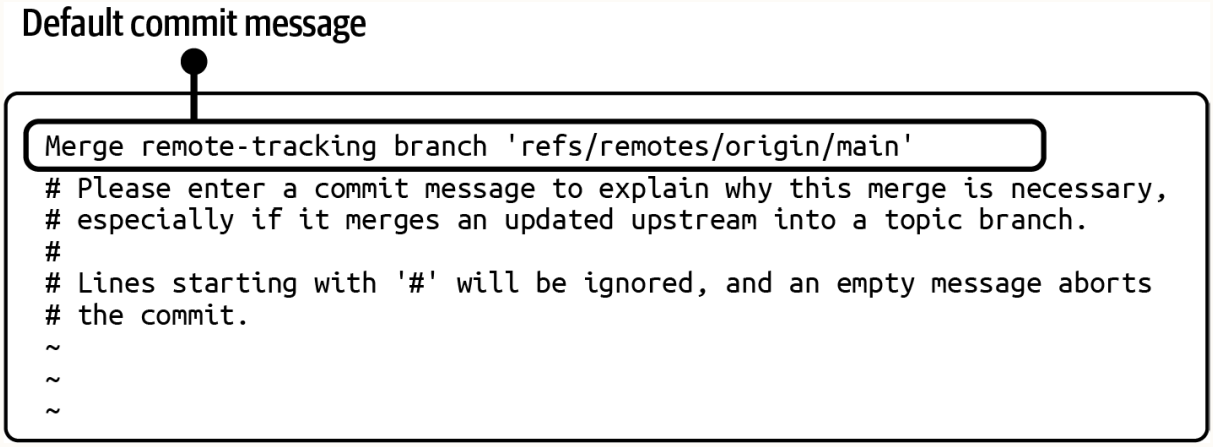


FIGURE 9-5

An example of a default commit message drafted by Git and presented in Vim in the command line

For the purposes of this example, we'll simply cover how to accept and save the default commit message. To do that you must press the Escape key, type a colon (:), then press the W key, the Q key, and the Enter key one after another in order, as seen in Figure 9-6. The w stands for "write" in Vim, which means save, and the q stands for "quit," which means exit. (If anything goes wrong, you can press Escape to try again.)

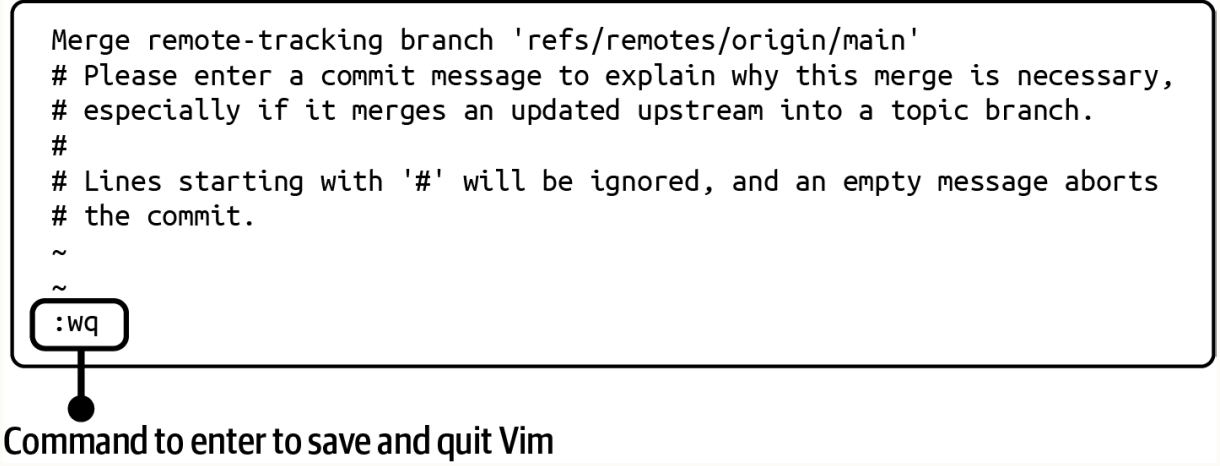


FIGURE 9-6

The command to enter to save and quit Vim

This saves the commit message and exits the Vim command line text editor. These instructions will be repeated in every Follow Along that requires you to exit Vim.

Now that we've covered some basics of how to work with Vim, your friend is ready to carry out the two steps to incorporate the remote changes into their local branch.

EXECUTING THE THREE-WAY MERGE

Having read the error message Git presented, your friend is now going to fetch the changes from the remote main branch into their local repository. This will update the origin/main remote-tracking branch. Then they will merge the origin/main remote-tracking branch into their local main branch.

Go to Follow Along 9-10 to carry out the two steps to incorporate the remote changes into the local main branch in the friend-rainbow repository.

[FOLLOW ALONG 9-10]

1. friend-rainbow $ **git fetch**

   remote: Enumerating objects: 4, done.
   remote: Counting objects: 100% (4/4), done.
   remote: Compressing objects: 100% (2/2), done.
   remote: Total 3 (delta 0), reused 3 (delta 0), pack-reused 0
   Unpacking objects: 100% (3/3), 290 bytes | 145.00 KiB/s, done.
   From github.com:gitlearningjourney/rainbow-remote
   6987cd2..7f0a87a main -> origin/main

2. friend-rainbow $ **git merge origin/main**

3. The command line will enter Vim after the git merge command is executed. The default commit message that Git drafts for you will either be Merge remote-tracking branch 'refs/remotes/origin/main' or Merge remote-tracking branch 'origin/main'. To accept the default message in the Vim editor and exit the editor, you must press the Escape key, type **:wq**, and then press the Enter key. You will see the following output:

   Merge made by the 'ort' strategy.
   othercolors.txt | 1 +
   1 file changed, 1 insertion(+)
   create mode 100644 othercolors.txt

[FOLLOW ALONG 9-10]

4 friend-rainbow $ **git log**

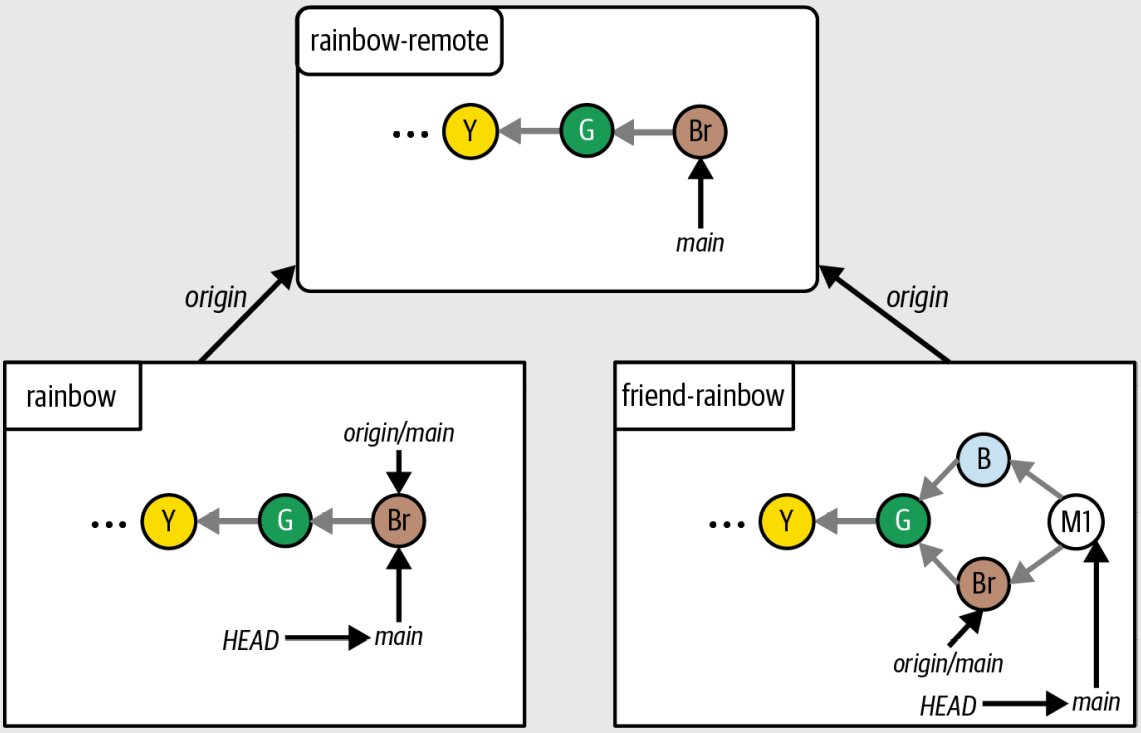
```
commit 225839938563c7458af81daca7beb782dfcbfb27 (HEAD -> main)
Merge: 342bbfc 7f0a87a
Author: annaskoulikari <gitlearningjourney@gmail.com>
Date: Sat Feb 19 13:05:46 2022 +0100
Merge remote-tracking branch 'refs/remotes/origin/main'
commit 342bbfc96bb03053f23ea7f7564ca207c58ceab2
Author: annaskoulikari <gitlearningjourney@gmail.com>
Date: Sat Feb 19 13:00:56 2022 +0100
blue
commit 7f0a87a318e50638eec50a484bf8dfa76b76d08e (origin/main,
origin/HEAD)
Author: annaskoulikari <gitlearningjourney@gmail.com>
Date: Sat Feb 19 12:46:29 2022 +0100
brown
```

What to notice:

- In step 3, the git merge output says Merge made by the 'ort' strategy. This indicates that this was a three-way merge instead of a fast-forward merge. (If you have an older version of Git, your output may say that the merge was made by the “recursive” strategy; this is also correct.)
- Git drafted a default commit message for you which was either Merge remote-tracking branch 'refs/remotes/origin/main' or Merge remote-tracking branch 'origin/main'. Both are correct.
- In step 4, the git log output shows the merge commit 2258399 and lists the two parent commits in the Merge field: 342bbfc (the blue commit) and 7f0a87a (the brown commit). The commit hashes for your commits will be different from the ones in this book, because commit hashes are unique.

Visualize it 9-12 shows the state of the Rainbow project after Follow Along 9-10.

[VISUALIZE IT 9-12]



The Rainbow project after your friend fetches the brown commit and carries out a three-way merge

What to notice:

- In the friend-rainbow repository, we illustrate the merge commit as M1 and show how it ties the two development histories together.

The M1 merge commit has two parent commits. Recall that in Chapter 4, you used the command git cat-file -p <commit_hash> to view the parent commits of a commit. In the upcoming Follow Along, your friend will use the same command, passing in the commit hash of the M1 merge commit, to see its parent commits (represented by their commit hashes). To retrieve the commit hash of the M1 merge commit, you can use the output of the git log command. Recall that you must use the commit hash of *your* M1 merge commit, not the commit hash in this book (because commit hashes are unique).

After that, your friend will push their changes to the remote repository to make sure it's updated. Go to Follow Along 9-11 to perform these actions now.

[FOLLOW ALONG 9-11]

1. Retrieve the commit hash for the M1 merge commit (you can copy this from the git log output in Follow Along 9-10). You must pass this commit hash as an argument to the git cat-file -p command in step 2 of this Follow Along. You may copy and paste the entire commit hash or just enter the first seven characters, as shown here.

2. friend-rainbow $ **git cat-file -p 2258399**

```
tree 45330906e6041a0cd07849617a25443a9a5b08bd
parent 342bbfc96bb03053f23ea7f7564ca207c58ceab2
parent 7f0a87a318e50638eec50a484bf8dfa76b76d08e
author annaskoulikari <gitlearningjourney@gmail.com> 1645272346 +0100
committer annaskoulikari <gitlearningjourney@gmail.com> 1645272346
+0100
Merge remote-tracking branch 'refs/remotes/origin/main'
```

3. friend-rainbow $ **git push**

```
Enumerating objects: 9, done.
Counting objects: 100% (8/8), done.
Delta compression using up to 4 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (5/5), 586 bytes | 586.00 KiB/s, done.
Total 5 (delta 1), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To github.com:gitlearningjourney/rainbow-remote.git
7f0a87a..2258399 main -> main
```

4. friend-rainbow $ **git log**

```
commit 225839938563c7458af81daca7beb782dfcbfb27 (HEAD -> main,
origin/main, origin/HEAD)
Merge: 342bbfc 7f0a87a
Author: annaskoulikari <gitlearningjourney@gmail.com>
Date: Sat Feb 19 13:05:46 2022 +0100
Merge remote-tracking branch 'refs/remotes/origin/main'
commit 342bbfc96bb03053f23ea7f7564ca207c58ceab2
Author: annaskoulikari <gitlearningjourney@gmail.com>
Date: Sat Feb 19 13:00:56 2022 +0100
blue
```

[FOLLOW ALONG 9-11]

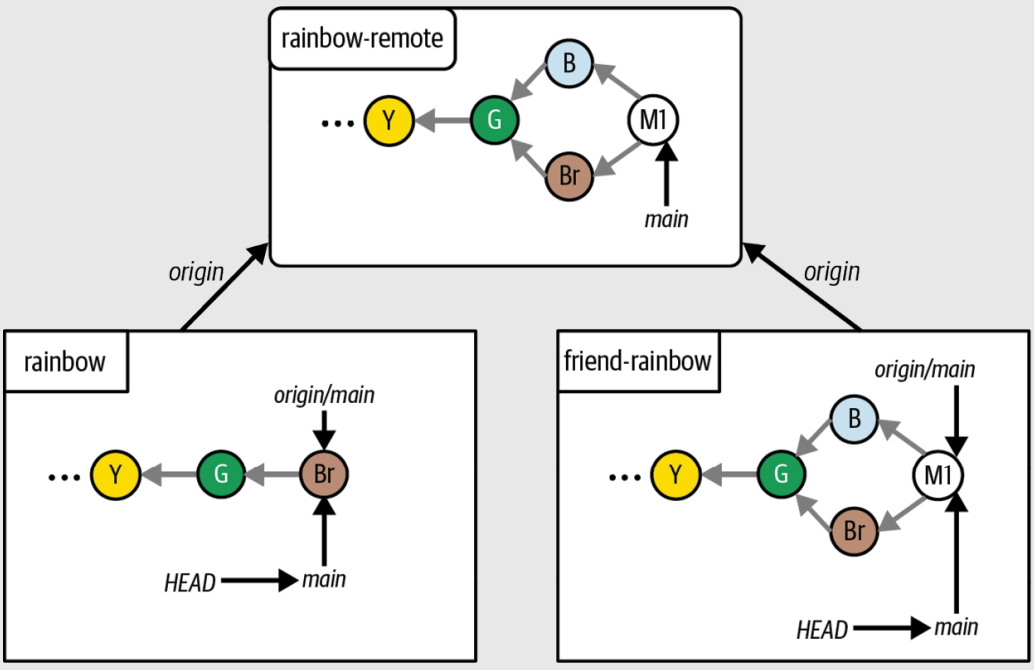
5 Go to the rainbow-remote repository on your hosting service and refresh the page. The merge commit should be there.

What to notice:

- In step 2, the git-cat file -p output shows the two commits listed as parents: 342bbfc96bb03053f23ea7f7564ca207c58ceab2 (the blue commit) and 7f0a87a318e50638eec50a484bf8dfa76b76d08e (the brown commit).
- In step 5, you can see that the merge commit is in the remote repository.

These observations are illustrated in Visualize it 9-13.

[VISUALIZE IT 9-13]



The Rainbow project after your friend pushes their changes to the remote repository

The M1 merge commit is now in the friend-rainbow repository and the rainbow-remote repository. The next step is for you to sync your rainbow repository with the remote

repository, updating your local main branch with the merge commit as well. To do that, you will learn about pulling in Git.

Pulling Changes from a Remote Repository

Up until now, in the Rainbow project, when you wanted to update your local repository with changes from the remote repository you did it in two steps: first you fetched the data from the remote repository (using the git fetch command), and then you merged the data (using the git merge command) into the local branch. Pulling data allows you to do both in one go.

In Git, we use the term *pull* or *pulling* to refer to the process of fetching data from a remote repository and integrating it into a branch in a local repository in one go, and the command we use to do it is git pull. If you don't have an upstream branch defined for your local branch, then you must specify the shortname of the remote repository and the name of the branch that you want to update. If you do have an upstream branch defined, you can just use git pull without any arguments.

[SAVE THE COMMAND]

git pull <***shortname***> <***branch_name***>

> Fetch and integrate changes from the <shortname> remote repository for the specified <branch_name>

git pull

> If an upstream branch is defined for the current branch, fetch and integrate changes from the defined upstream branch

There's one more thing you need to know about the git pull command. As you've learned, in Git there are two ways to integrate changes: merging and rebasing. Which method the git pull command uses will depend on whether the development histories of the branches have diverged and, if so, on the option you choose when entering the command:

- If the development histories of the local branch and remote branch in a git pull have *not* diverged, then by default a fast-forward merge will occur.
- If the development histories of the local branch and the remote branch in a git pull *have* diverged, then you must tell Git whether you want to integrate the changes by merging or rebasing (otherwise, you'll get an error). To tell Git to integrate the changes by merging, you must pass in the --no-rebase option. To tell Git to integrate the changes by rebasing, you must pass in the --rebase option.

As illustrated in Figure 9-7, the git pull command effectively combines the git fetch command and either the git merge or the git rebase command.



FIGURE 9-7

The git pull command fetches and integrates changes in one go

Now the question is, when should you fetch and integrate changes in two steps by using the git fetch command and then either the git merge or the git rebase command, and when should you carry out both of those steps in one go by just using the git pull command?

It is common for Git users to use the git pull command when the development histories of the local and remote branches have not diverged, and therefore a simple fast-forward merge will happen. If the development histories of the local and remote branches have diverged, Git users often prefer to use the git fetch command then choose whether to rebase or merge in a separate step. By carrying out the process in two steps, they give themselves more time to look at what is going to change in their local branch and to prepare for the integration process.

In this book you will follow this common practice, and therefore you will use only the git pull command when a fast-forward merge will update your local branch.

Go to Follow Along 9-12 to practice using the git pull command by pulling the changes from the remote main branch to the local main branch in the rainbow repository.

[FOLLOW ALONG 9-12]

1. rainbow $ **git pull**

```
remote: Enumerating objects: 9, done.
remote: Counting objects: 100% (8/8), done.
remote: Compressing objects: 100% (3/3), done.
remote: Total 5 (delta 1), reused 5 (delta 1), pack-reused 0
Unpacking objects: 100% (5/5), 566 bytes | 141.00 KiB/s, done.
From github.com:gitlearningjourney/rainbow-remote
7f0a87a..2258399 main -> origin/main
Updating 7f0a87a..2258399
Fast-forward
rainbowcolors.txt | 3 ++-
1 file changed, 2 insertions(+), 1 deletion(-)
```

2. rainbow $ **git log**

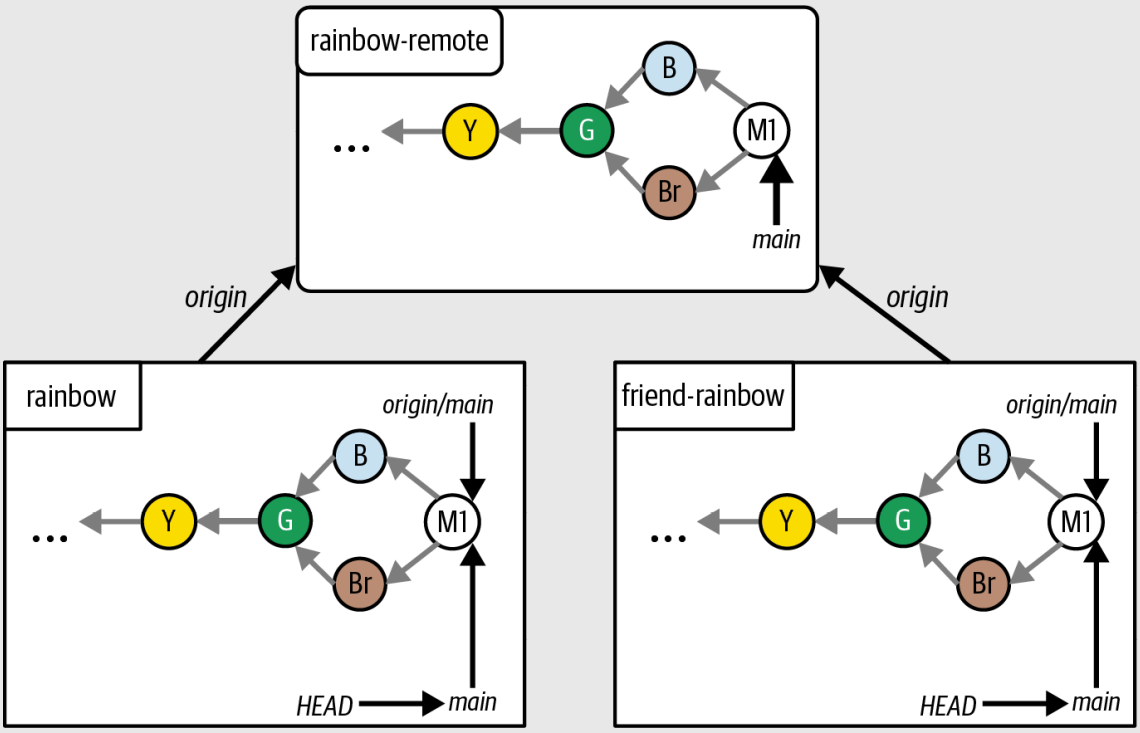
```
commit 225839938563c7458af81daca7beb782dfcbfb27 (HEAD -> main, origin/main)
Merge: 342bbfc 7f0a87a
Author: annaskoulikari <gitlearningjourney@gmail.com>
Date: Sat Feb 19 13:05:46 2022 +0100
Merge remote-tracking branch 'refs/remotes/origin/main'
commit 342bbfc96bb03053f23ea7f7564ca207c58ceab2
Author: annaskoulikari <gitlearningjourney@gmail.com>
Date: Sat Feb 19 13:00:56 2022 +0100
blue
```

What to notice:

- The first half of the git pull output in step 1 shows the data is being fetched or downloaded. The second half of the output indicates that a Fast-forward merge was carried out.
- The git log output in step 2 shows that you now have the blue commit and the merge commit in the rainbow repository, and the origin/main remote-tracking branch and the main branch have been updated to point to the merge commit.

Visualize it 9-14 illustrates these observations.

[VISUALIZE IT 9-14]



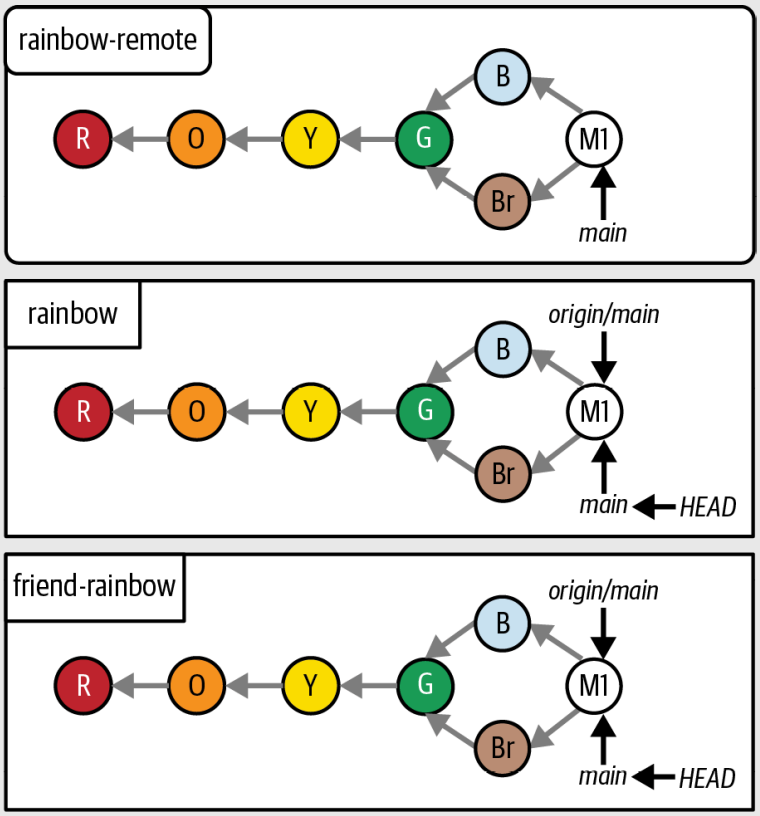
The Rainbow project after you pull changes from the remote repository into the main branch in the rainbow repository

All three repositories in the Rainbow project are now in sync.

State of the Local and Remote Repositories

Visualize it 9-15 shows the state of the local and remote repositories in the Rainbow project with all the commits that were made from Chapter 1 all the way to the end of this chapter.

[VISUALIZE IT 9-15]



The Rainbow project at the end of Chapter 9 with all the commits since Chapter 1

Summary

In this chapter you learned about three-way merges. To see how they work, you made changes in the Rainbow project that resulted in the histories of the local main branch in the friend-rainbow repository and the remote main branch diverging.

In the process of creating these divergent histories you learned how to define upstream branches, and you saw how after defining them you were able to use commands such as git push without specifying any additional arguments.

You also learned an important detail about editing files in your working directory multiple times between commits: namely, every time you make a change to a file and save it, Git will interpret this as a new version of the file and will mark the file as modified. To include the latest version of the file in the next commit, you will need to add that version of the file to the staging area.

When it came time to carry out the three-way merge, you learned about the Vim command line text editor and covered the basics of how to save Git's proposed commit message and exit Vim. At the end of the merge process, you saw how the merge commit tied the two divergent development histories together.

Finally, you learned about the process of pulling and how it allows you to carry out the two steps of incorporating the changes from a remote repository into a local repository in one go.

The three-way merge you carried out in this chapter didn't have any merge conflicts. In Chapter 10, you're going to learn about merge conflicts and how to resolve them.

Previous chapter
‹ Chapter 8. Cloning and Fetching

Next chapter
Chapter 10. Merge Conflicts ›